Algorithm Engineering
Woche 2, Aufgabe 3

Jan Vogt

Coding Warmup (Folie 20)

Alle Programme liegen im selben Ordner und wurden auf meinem Laptop (Apple M3) mit `g++-16 -fopenmp` kompiliert und ausgeführt.

1. Race Condition mit std::mutex beheben

Auf Folie 13 schreiben alle Threads am Ende gleichzeitig in die gemeinsame Variable `pi`. `pi += ...` besteht aber aus Lesen, Addieren und Schreiben. Wenn zwei Threads gleichzeitig lesen, geht der Beitrag von einem der beiden verloren. In `race_condition.cpp` wird das Update deshalb mit einem `std::mutex` geschützt:

```
lock_guard<mutex> lock(pi_mutex);
pi += sum_local * 4 * width;
```

Mit 100.000.000 Schritten ist der Fehler auch ohne Mutex in 20 Läufen nie aufgetreten, weil die Threads zu unterschiedlichen Zeiten fertig werden. Deshalb habe ich beide Versionen mit nur 1000 Schritten jeweils 2000 mal laufen lassen:

Version	Anzahl	Ergebnis
ohne Mutex	1960	3.1415927369231262
	3	3.1415927369231271
	18	2.3564443034735985
	18	2.3559443019110908
	1	2.3554448015985905
mit Mutex	1994	3.1415927369231262
	6	3.1415927369231271

Ohne Mutex war das Ergebnis in 37 Läufen falsch. Die falschen Werte liegen alle bei ungefähr $\frac{3}{4}\pi$, es ist also genau der Beitrag von einem Thread verloren gegangen. Mit Mutex war das Ergebnis immer richtig. Dass sich in manchen Läufen die letzte Stelle unterscheidet, liegt daran, dass die Threads ihre Teilsummen in unterschiedlicher Reihenfolge addieren.

2. Schedule für das Denoising-Programm

Auf Folie 16 braucht Bild `i` eine Arbeit proportional zu i^2. Ohne Schedule verteilt OpenMP die Schleife statisch in vier gleich große Blöcke. Weil die Schleife rückwärts läuft, bekommt Thread 0 die Bilder 63 bis 48 und hat damit viel mehr Arbeit als die anderen. Gemessen habe ich mit 4 Threads (jeweils 5 Läufe):

Schedule	Zeit [s]
ohne Schedule	0.503 bis 0.507
`static`	0.504 bis 0.506
`static, 1`	0.240 bis 0.320
`dynamic`	0.224 bis 0.237
`dynamic, 1`	0.224 bis 0.230
`dynamic, 4`	0.225 bis 0.239
`guided`	0.505 bis 0.513

Mit `schedule(dynamic, 1)` holt sich jeder Thread immer das nächste Bild, sobald er fertig ist. Dadurch ist das Programm etwa 2,2 mal schneller. `guided` bringt hier nichts, weil es am Anfang große Blöcke verteilt und der erste Block genau die schwersten Bilder enthält.

3. Welcher Schedule erzeugt das Muster?

Das Muster auf Folie 20 entsteht mit `schedule(static, 22)`. Die 32 Iterationen werden in Blöcke mit 22 Iterationen aufgeteilt. Thread 0 bekommt den ersten Block (0 bis 21), Thread 1 den Rest (22 bis 31) und Thread 2 und 3 bekommen gar nichts. Das habe ich mit `schedule_clause.cpp` getestet und es kam in jedem Lauf genau diese Aufteilung heraus. Mit `dynamic, 22` entstehen zwar dieselben zwei Blöcke, aber welcher Thread sie bekommt, ändert sich bei jedem Lauf.